DLDCA Lab 4: Simulator
Weeks 2&3

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to use a state-of-the art microarchitectural simulator (Champsim) to see the effects of different microarchitectural policies on different workloads.

2 Necessary tools

We shall be using `make` and `g++` in order to compile C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ make
make: *** No targets specified and no makefile found.  Stop.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has the `traces` installed, this can be checked by running:

```
$ ls $CS231_TRACE_DIR
bwaves.xz  gcc.xz  mcf.xz  sssp.gz
```

You must also ensure that Champsim has been installed and setup in the Desktop folder of your lab machine, there must be a folder on your desktop called `Champsim`. You can ensure it is correct by running the following from your desktop:

```
ls Champsim/
bin  branch  btb  champsim_config.json  CITATION.cff  config.sh
inc  LICENSE  Makefile  obj  prefetcher  replacement  run_champsim.sh  src  storage  tracer
```

3 Structure of submission/templates

```
your-roll-no/
|- task1/
    |- cache.cc
|- task2/
    |- cache.cc
```

```
        |- deadblock.cc
        |- sampler.h
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of two subtasks. These tasks are expected to be submitted out-lab.

Some background

How to build Champsim?

To build champsim, all you have to do is run `./config.sh champsim_config.json && make` This builds the champsim binary with the required cache management policy. You may modify the executable name by changing the `executable_name` inside `champsim_config.json` to something other than `bin/lru`.

How to run Champsim?

We have given you a tool to run Champsim, the tool takes in two arguments:

1. `TRACE_FILE`: This is the workload which is run on the simulator, you have been given four workloads which are found in the `$CS231_TRACE_DIR` folder
2. `BINARY`: This is the specific champsim binary which contains the simulator on which to run the workload

An example run of champsim looks like:

```
 $ bash run_champsim.sh $CS231_TRACE_DIR/sssp.gz random

*** ChampSim Multicore Out-of-Order Simulator ***
...
```

Task 1: An Exclusive Last Level Cache

An LLC is said to be *exclusive* if for every memory address in the LLC, it is not present in either the L1D or L2 Caches. The advantage of having an exclusive last level cache is that you have a larger effective cache capacity. By default, the LLC in Champsim is *non-inclusive.* This means, that a block gets filled into the LLC on two events:

1. When a response comes to the LLC from the DRAM
2. When the L2 Cache issues a writeback to the LLC

To implement an exclusive LLC, we need to make the following changes:

1. When a response comes from the DRAM, do not fill into the LLC: This is because the memory address will anyway be filled into the L1D and L2 Caches, therefore, we do not fill it into the LLC

2. The L2 Cache must writeback clean blocks to the LLC: This is because otherwise clean blocks will never get filled into the LLC. By default, only dirty blocks are written back by the L2 to the LLC.
3. On an LLC Hit, invalidate the block that received the hit: This is because you will only get a LLC hit if the L2 Cache has requested for that memory address, meaning the memory will be filled into the L2, thus, it should be removed from the LLC

Remember, the code for handling a response is in the `handle_fill` function, that for handling a request is in the `handle_read` function and writeback packets are generated in `filllike_miss`. If implemented correctly, you should see a good performance boost in `mcf` (around 15-17%) and `sssp` (around 17-20%). Also please note that you should only be seeing a speedup if your LLC MPKI (Misses per kilo instruction) has decreased. If your LLC MPKI increases but you get a speedup, it is wrong.

Task 2: Sampling Dead Block Predictor for the Exclusive LLC

A cache block is defined to be *dead-on-arrival* if it gets evicted before it receives a hit in the cache. Such a block is useless and filling it in the cache does not give any benefit in terms of performance.

Identifying dead blocks can be very useful for performance due to the fact that bypassing (skipping) the cache fill of a dead block does not harm performance in anyway and can infact benefit performance by creating more empty space for useful blocks.

The microarchitectural community has proposed various interesting designs for deadblock predictors. In this lab you will implementing a variant of the Sampling dead block predictor.

The predictor has multiple components:

- Sampling cache: A smaller cache that maintains tags and PC's of the currently tracked elements (Not a part of the lab)
- Dead Block Counter: Three different tables which are indexed by the lower 14 bits of the PC which are hashed by three different hash functions
- Bypassing mechanism which bypasses fills from L2 to LLC (at the time of L2 eviction)
- LLC Replacement Policy which priortizes dead Blocks to be kicked from the cache.

The basic intution for the predictor is the fact that if instruction at PC 'x' has majorly only brought deadblocks into the cache then it will continue to bring in deadblocks in the future. Thus we intend to maintain a (number of dead blocks - number of live blocks) counter for each PC. This is done by incrementing the counter when a block corresponding to that entry is found to be dead (evicted) and by decrementing the counter if a block corresponding to that entry is found to be live (got a hit). For each block we decide it is dead if the corresponding PC is 'dead' (its counter is higher than a threshold).

Note that this is done in an exclusive hierarchy and thus any block evicted from LLC is dead (it would have been invalidated on a hit otherwise). Also note that inorder to maintain these counters we would need the PC information of a block in the LLC (the PC of the instruction that brought it into the cache hierarchy from DRAm) at the time of a hit or a eviction. This is the role of the sampling cache. In this lab however to simplify the implementation we just maintain the PC of all the cache lines in the `BLOCK` struct itself. Thus make sure that PC information is propagated to all the blocks in the LLC.

Some issues:

- **Too many PC's:** There are too many PC's for us to easily maintain a table of all of them. Thus we use the lower 14 bits of the PC to index into a smaller table and any 2 PC's with the same lower 14 bits share a counter.

- **PC Collision:** An obvious issue with the above scheme is as follows. Consider PC's x and y which share the same lower 14 bits. If x primarily brings in dead blocks and y is primarily brings in live blocks, what would the share counter decide? This is referred to as destructive interference. To solve this problem, we use three different hashes to index into three different tables of counters. The intuition is that x and y colliding on all three counters is very unlikely.

In this task you are expected to build the (skewed) PC counter table to maintain dead - live block counts. This count is also refered to as confidence. We declare a block to be dead if the sum of confidence1,confidence2 and confidence3 is greater than a threshold.

In this case the confidence counters are 2-bit counters and the sum ranges from 0-9. The threshold is 8.

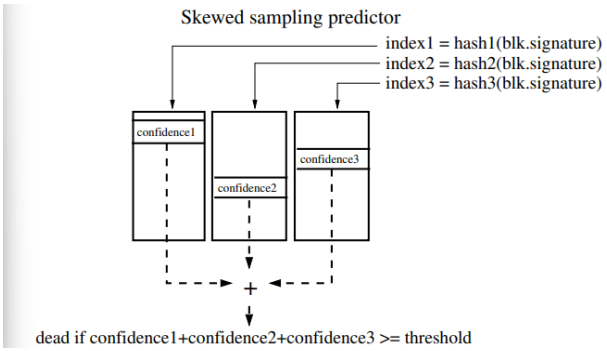

Figure 1: Predictor tables and confidence counters

SubTasks:

1. You have been provided with `sampler_template.h` to implement the PC based predictors. Fill in the definitions of the appropriate functions to complete this task.

2. Use the predictor class implemented in the previous part and modify cache.cc to instantiate and call the appropriate functions for the predictor.

   Now there is a problem with this. When do we update the predictor? Do we update on every LLC eviction/hit? The answer is no since that is expensive. Infact if we update it only on a subset of evictions/hits it is sufficient. To do this randomly sample (16 sets per 1024 sets in the LLC) to be 'observers'. Our predictor is trained only on LLC evictions/hits of blocks that were present in the observer sets. The observer sets have been initialized for you. Call the update function for the predictor only when the evicted block / hit block belongs to these sets.

3. Further you would also have to bypass L2 to LLC fills (on L2 evictions) when a fill is corresponding to a block predicted to be dead. Thus modify cache.cc to query the predictor before every L2 to LLC fill and bypass fills predicted to be dead (fills that create deadblocks)

4. The last subtask would be to use the deadblock information to enhance the lru replacement policy (for LLC). The modification you are expected to do is to check if any of the ways in the LLC are occupied by blocks predicted to be dead. If so they are chosen as replacement victims, else fall back to lru. For this purpose you are given a template file `deadblock_template.cc` which has the predictor instance and a basic lru implementation.

To use the templates copy them into the appropriate champsim folders. Header files to '/inc', the cache.cc file to '/src' and for the replacement policy create a new replacement policy called 'deadblock' and a file inside it 'deadblock.cc' and copy the template into that file.